

LAPORAN PRAKTIKUM

BIG DATA

Modul 3:

ETL/ELT Pipeline Akuisisi, Pembersihan, dan Transformasi Data

Oleh

GHAZI FADHLULLAH

NIM. 2311532012



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Link github :

https://github.com/GhaziFadhlullah/BigData26_kelasB_2311532012_GHAZIFADHLULLAH/tree/main/PRATIKUM%203

Link drive :

https://drive.google.com/drive/folders/15xaZ9oo95Cj6quA7nfmgGZr4uNTtX89u?usp=drive_link

A. LANGKAH DAN HASIL PRAKTIKUM

K-1: Setup Lingkungan & Library

Pada blok ini seluruh library yang diperlukan diimpor dan path direktori lapisan_mentah serta lapisan_terkurasi dibuat. Variabel global BULAN_UNDUH mendefinisikan periode data yang akan diunduh (dua bulan: Jan–Feb 2023).

```
*** pandas : 2.2.3
    numpy : 2.1.3
    pyarrow : 23.0.1
    requests : 2.32.4
    sklearn : 1.6.1
    sqlalchemy : 2.0.54
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content
```

K-2: Akuisisi Data Taksi NYC TLC

Fungsi unduh_aman() mengimplementasikan unduhan file Parquet dengan mekanisme retry (3 kali) dan atomic write (menulis ke file sementara dahulu, lalu rename). Hal ini memastikan file tidak korup meskipun proses terganggu di tengah jalan. Data zona taksi (taxi_zone_lookup.csv) diunduh dan dibaca dengan compression='gzip' karena file disimpan dalam format gzip.

```

cache ditemukan: yellow_tripdata_2023-01.parquet
...
[unduh trip] 0.00 s
cache ditemukan: taxi_zone_lookup.csv
[unduh zona] 0.00 s
yellow_tripdata_2023-01.parquet -> 45.46 MB
taxi_zone_lookup.csv -> 0.0 MB
[baca trip] 0.73 s
[lineage] trip: 3,066,766 baris
[lineage] zona: 265 baris

Bentuk data trip : (3066766, 10)
Bentuk data zona : (265, 4)

```

	LocationID	Borough	Zone	service_zone
0	1	EWR	Newark Airport	EWR
1	2	Queens	Jamaica Bay	Boro Zone
2	3	Bronx	Allerton/Pelham Gardens	Boro Zone
3	4	Manhattan	Alphabet City	Yellow Zone
4	5	Staten Island	Arden Heights	Boro Zone

K-3: Akuisisi Data Cuaca (Open-Meteo API)

Data cuaca harian (suhu, curah hujan, kecepatan angin) diambil dari Open-Meteo API untuk koordinat New York City. Data JSON dikonversi menjadi DataFrame dan disimpan ke lapisan_mentah sebagai cuaca_nyc_2023.parquet.

```

[ambil cuaca] 0.40 s
[lineage] cuaca: 744 baris

Baris cuaca      : 744
Jam diharapkan   : 744
Selisih          : 0 jam ✓

```

	jam_mulai	suhu_c	hujan_mm
0	2023-01-01 00:00:00	10.9	1.0
1	2023-01-01 01:00:00	10.6	1.0
2	2023-01-01 02:00:00	10.6	0.1
3	2023-01-01 03:00:00	10.5	0.0
4	2023-01-01 04:00:00	9.8	0.0

K-4: Profiling Kualitas Data — Enam Dimensi

Fungsi `profil_kualitas()` menghitung semua enam dimensi kualitas data secara otomatis. Hasilnya disajikan dalam bentuk dictionary yang memudahkan perbandingan sebelum dan sesudah pembersihan.

```
K-4 siap (tidak aktif - K-3 berhasil).
```

K-5: Penanganan Missing Value

Empat strategi diterapkan pada kolom yang berbeda: (1) Drop pada baris yang trip_distance atau fare_amount-nya null; (2) Global median untuk passenger_count; (3) Global mode untuk RatecodeID; (4) Per-group median untuk kolom amount berdasarkan PULocationID.

```
... Tabel dibuat: ['tarif_referensi', 'trip_operasional']
      payment_type  jumlah  rata_total
0                1  226967   31.339172
1                2   66504   25.218836
2                4    2082   26.611114
3                3    1553   22.089691

Dibaca bertahap: 300,000 baris
[lineage] tarif_referensi: 6 baris
```

K-6: Deteksi & Penanganan Outlier

IQR method digunakan untuk trip_distance dan fare_amount. Z-score method digunakan untuk total_amount. Nilai outlier di-winsorize (di-clip ke batas IQR). Baris yang tidak masuk akal (trip_distance ≤ 0, fare_amount < 0) dihapus.

```
=== PROFIL KOLOM ===
      kolom      tipe  persen_hilang  nilai_unik      contoh
...  tpep_pickup_datetime  datetime64[us]    0.000  1610975 2023-01-01 00:32:10
    tpep_dropoff_datetime  datetime64[us]    0.000  1611319 2023-01-01 00:40:36
      passenger_count      float64    2.339      10      1.0
      trip_distance      float64    0.000   4387     0.97
      PULocationID      int64    0.000    257    161
      DOLocationID      int64    0.000    261    141
      payment_type      int64    0.000     5     2
      fare_amount      float64    0.000   6873     9.3
      tip_amount      float64    0.000   4036     0.0
      total_amount      float64    0.000  15871    14.3
      durasi_minut      float64    0.000   8579    8.433333

=== LAPORAN KUALITAS DATA ===
      aturan  lulus  gagal  persen_gagal
kelengkapan: passenger_count ada 2995023  71743    2.339
validitas: jarak 0-100 mil 3020816  45950    1.498
konsistensi: durasi 1-180 menit 3030383  36383    1.186
validitas: total_amount > 0 3040994  25772    0.840
konsistensi: total >= fare 3041743  25023    0.816
ketepatan waktu: dalam Jan 2023 3066718    48    0.002
keunikan: baris tidak duplikat 3066766    0    0.000
validitas: PULocationID dikenal 3066766    0    0.000
```

K-7: Multi-Source Join

Data taksi digabungkan dengan: (1) Zona taksi (PULocationID → Borough, Zone) menggunakan left join; (2) Data cuaca (tanggal pickup → suhu, hujan, angin) menggunakan left join. Parameter validate='many_to_one' memastikan tidak terjadi row explosion pada join.

```
Persen hilang 'passenger_count': 2.339%
```

```
=== PERBANDINGAN STRATEGI NILAI HILANG ===
```

strategi	n	mean	median	std
1_dibuang	2995023	1.3625	1.0	0.8961
2_isi_median	3066766	1.3541	1.0	0.8873
3_isi_modus	3066766	1.3541	1.0	0.8873
4_median_perjam	3066766	1.3541	1.0	0.8873

```
Duplikat penuh : 0
```

```
Duplikat pd kunci : 1
```

```
3,066,766 -> 3,066,765 baris (hapus 1 duplikat)
```

K-8: Feature Engineering

Fitur baru yang dibuat:

- `durasi_menit`: selisih dropoff dan pickup dalam menit.
- `speed_mph`: $\text{trip_distance} / (\text{durasi_menit} / 60)$, diclip agar tidak ∞ .
- `fare_per_mile`: $\text{fare_amount} / \text{trip_distance}$.
- `is_weekend`: 1 jika hari pickup adalah Sabtu atau Minggu.
- `is_rain`: 1 jika `curah_hujan` > 0 pada hari tersebut.

```
=== RINGKASAN OUTLIER ===
```

kolom	batas_bawah	batas_atas	outlier_iqr	outlier_z3	p99
<code>trip_distance</code>	-2.35	6.74	390244	67	20.06
<code>durasi_menit</code>	-9.66	35.08	170615	3175	57.25
<code>total_amount</code>	-4.55	48.65	371616	87248	101.94

```
3,066,765 -> 2,986,910 baris (2.60% dibuang)
```

```
Kunci zona unik? True
```

```
Setelah join zona : 2,986,910 -> 2,986,910 baris
```

```
Zona tak dikenal : 37,609
```

```
Setelah join cuaca : 2,986,910 -> 2,986,910 baris
```

```
Tanpa data cuaca : 37
```

K-9: Scaling & Encoding

Train/test split dilakukan sebelum scaling untuk mencegah data leakage. StandardScaler diterapkan pada fitur numerik (mean=0, std=1). MinMaxScaler diterapkan sebagai alternatif (range [0,1]). OneHotEncoder dengan handle_unknown='ignore' digunakan untuk fitur kategorikal Borough.

```
*** Mean data latih (harus ~0) : [-0.  0. -0.]
    Mean data uji   (# 0)   : [ 0.001  0.001 -0.009]

    Jumlah kolom hasil one-hot : 11
    Nama kolom (6 pertama)    : ['nama_pembayaran_Gratis', 'nama_pembayaran_Kartu kredit', 'nama_penba
```

K-10: Pipeline Idempoten & Output Parquet Terpartisi

Fungsi pipeline() mengintegrasikan seluruh langkah K-2 hingga K-9 secara berurutan. Bersifat idempoten: dapat dijalankan ulang dengan hasil yang sama. Fungsi validasi() memverifikasi skema kontrak output (tipe data, kolom wajib, tidak ada null pada kolom kritis). Output disimpan sebagai Parquet terpartisi per borough di lapisan_terkurasi.

```
✅ Validasi lulus: 2,986,910 baris x 17 kolom
[pipeline penuh] 10.39 s
[tulis parquet berpartisi] 4.48 s
Partisi yang terbentuk: ['borough_naik=Bronx', 'borough_naik=Brooklyn', 'borough_naik=EWR', 'boroug
✅ Validasi lulus: 2,986,910 baris x 17 kolom
Idempoten? True

✅ Semua artefak tersimpan di Drive.
```

K-11: PySpark Equivalent Pipeline

Pipeline diimplementasikan ulang menggunakan PySpark untuk demonstrasi skalabilitas. Broadcast join digunakan untuk tabel zona (kecil). Fungsi F.unix_timestamp() digunakan untuk menghitung durasi dari kolom TIMESTAMP_NTZ. Output disimpan sebagai Parquet dengan partitionBy('Borough').

```
... [spark: hitung baris bersih] 38.25 s

Baris bersih (Spark) : 2,986,910
Baris bersih (pandas) : 2,986,910
Selisih               : 0

+-----+-----+
|Borough|count|
+-----+-----+
|Manhattan|2659609|
|Queens|270315|
|Unknown|37609|
|Brooklyn|15724|
|Bronx|3074|
|NaN|340|
|Staten Island|211|
|EWR|28|
+-----+-----+
```

B. JAWABAN LATIHAN

Latihan 1: Mengapa atomic write penting pada `unduh_aman()`?

Atomic write memastikan bahwa file yang tersimpan selalu dalam kondisi lengkap dan valid. Tanpa atomic write, jika proses unduhan terhenti di tengah jalan (karena koneksi putus, proses di-kill, atau kegagalan sistem), file yang tersimpan sebagian akan tampak ada di direktori namun isinya korup/tidak lengkap. Pada langkah berikutnya, pembacaan file tersebut akan gagal dengan error atau menghasilkan data yang salah tanpa peringatan. Dengan pola `write-ke-tmp-lalu-rename`, `rename` bersifat atomik di level OS: baik file lama tetap utuh, atau file baru langsung menggantikannya secara sempurna — tidak ada kondisi di antaranya.

Latihan 2: Kapan per-group median lebih baik daripada global median?

Per-group median lebih baik ketika distribusi nilai berbeda secara signifikan antar kelompok. Pada dataset taksi, `tip_amount` di Manhattan sangat berbeda dengan di Bronx atau Queens karena perbedaan jarak rata-rata, tingkat ekonomi penumpang, dan faktor lokal lainnya. Mengisi dengan global median akan menciptakan bias sistematis: perjalanan di area dengan tip rendah akan diisi terlalu tinggi, dan sebaliknya. Per-group median mempertahankan karakteristik distribusi spasial data.

Latihan 3: Apa perbedaan IQR dan Z-score untuk deteksi outlier?

IQR (Interquartile Range) adalah metode non-parametrik yang tidak bergantung pada asumsi distribusi normal. IQR robust terhadap outlier itu sendiri karena menggunakan kuartil, bukan mean/std. Cocok untuk distribusi skewed seperti `fare_amount` yang memiliki ekor kanan panjang. Z-score adalah metode parametrik yang mengasumsikan distribusi normal. Z-score sensitif terhadap outlier karena mean dan std dipengaruhi oleh nilai ekstrem. Pada data taksi, IQR lebih sesuai untuk `fare_amount` dan `trip_distance` karena distribusinya tidak normal, sedangkan Z-score dapat digunakan pada `total_amount` setelah winsorizing awal.

Latihan 4: Mengapa `validate='many_to_one'` digunakan pada join?

Parameter `validate='many_to_one'` memerintahkan pandas untuk memverifikasi bahwa kunci join di tabel kanan (tabel zona, tabel cuaca) bersifat unik. Jika kunci kanan ternyata duplikat, pandas akan melempar `MergeError` sebelum join terjadi. Tanpa validasi ini, satu baris taksi bisa bergabung dengan banyak baris zona/cuaca, mengakibatkan row explosion — jumlah baris output bisa berlipat ganda secara diam-diam, menyebabkan perhitungan statistik yang salah dan laporan yang misleading.

Latihan 5: Jelaskan data leakage dan cara pencegahannya!

Data leakage terjadi ketika informasi dari data test 'bocor' ke dalam proses training, sehingga model terlihat performanya lebih baik dari sebenarnya. Contoh klasik: jika `StandardScaler` di-fit pada seluruh dataset (`train+test`), mean dan std yang digunakan

untuk normalisasi sudah 'melihat' data test. Model seolah tahu distribusi data test, dan hasil evaluasi menjadi optimistic bias. Pencegahan: selalu lakukan train/test split SEBELUM fit scaler/encoder. Scaler hanya di-fit pada data train (scaler.fit_transform(X_train)), kemudian di-transform pada data test (scaler.transform(X_test)).

C. JAWABAN TUGAS

Tugas 1: Deteksi Duplikat Multi-Bulan

Pendekatan yang digunakan untuk mendeteksi duplikat lintas bulan:

- Semua file Parquet dari dua bulan (Jan–Feb 2023) dibaca sekaligus menggunakan `pd.read_parquet()` pada folder partisi.
- Duplikat diidentifikasi berdasarkan kombinasi kolom kunci: `tpep_pickup_datetime`, `tpep_dropoff_datetime`, `PULocationID`, `DOLocationID`, dan `total_amount`.
- `pd.DataFrame.duplicated(keep=False)` menandai semua baris duplikat (bukan hanya yang pertama).

Hasil: dari total baris yang diproses, [N] baris teridentifikasi sebagai duplikat ([X]%). Duplikat terjadi karena [alasan, misal: overlap data antara file bulan yang berbeda atau data entry yang dimasukkan dua kali].

```
... cache ditemukan: yellow_tripdata_2023-01.parquet
✅ Validasi lulus: 2,986,910 baris x 17 kolom
[pipeline 2023-01] 17.81 s
Bulan 2023-01: 2,986,910 baris tersimpan
cache ditemukan: yellow_tripdata_2023-07.parquet
[ambil cuaca 2023-07] 0.40 s
✅ Validasi lulus: 2,817,657 baris x 17 kolom
[pipeline 2023-07] 6.98 s
Bulan 2023-07: 2,817,657 baris tersimpan

=== RINGKASAN DUA BULAN ===
    2023-01: 2,986,910 baris
    2023-07: 2,817,657 baris
File parquet ditemukan: 0
['bulan=2023-01', 'bulan=2023-07']
Total baris gabungan : 5,804,567
Duplikat antar-bulan : 0 (✅ aman)
```

Tugas 2: Laporan Kualitas Data Bulanan

Laporan kualitas data dihitung per bulan menggunakan fungsi `buat_laporan_kualitas()`. Iterasi dilakukan dengan menggunakan `.replace()` untuk menghindari error `MonthBegin` pada pandas (pengganti `.floor('MS')` yang tidak kompatibel dengan non-fixed frequency).

=== Laporan kualitas komparatif (% baris GAGAL per aturan) ===

	aturan	persen_gagal_2023-01	persen_gagal_2023-02
...	kelengkapan: passenger_count ada	2.339	2.636
	ketepatan waktu: dalam bulan ini	100.000	100.000
	keunikan: baris tidak duplikat	0.000	0.000
	konsistensi: durasi 1-180 menit	1.186	1.191
	konsistensi: total >= fare	0.816	0.849
	validitas: PULocationID dikenal	0.000	0.000
	validitas: jarak 0-100 mil	1.498	1.414
	validitas: total_amount > 0	0.840	0.874

Disimpan: /content/drive/MyDrive/BigData/Praktikum3/lapisan_terkurasi/laporan_kuali

	aturan	persen_gagal_2023-01	persen_gagal_2023-02
0	kelengkapan: passenger_count ada	2.339	2.636
1	ketepatan waktu: dalam bulan ini	100.000	100.000
2	keunikan: baris tidak duplikat	0.000	0.000
3	konsistensi: durasi 1-180 menit	1.186	1.191
4	konsistensi: total >= fare	0.816	0.849
5	validitas: PULocationID dikenal	0.000	0.000
6	validitas: jarak 0-100 mil	1.498	1.414

Tugas 3: Sumber Data Keempat — Kualitas Udara NYC (OpenAQ API)

Sumber data keempat yang ditambahkan adalah data indeks kualitas udara (AQI) dari OpenAQ API (<https://api.openaq.org>). OpenAQ menyediakan data kualitas udara real-time dan historis dari ribuan stasiun monitoring di seluruh dunia, termasuk NYC.

Proses Akuisisi Data:

- Endpoint: <https://api.openaq.org/v2/measurements> dengan parameter city=New York, parameter=pm25.
- Data yang diambil: tanggal, nilai PM2.5 (partikel halus $\mu\text{g}/\text{m}^3$) per hari.
- Diagregasi menjadi rata-rata harian per kota, disimpan sebagai aqi_nyc_2023.parquet.

Cara Integrasi ke Pipeline:

- Join berdasarkan kolom tanggal (date dari tpep_pickup_datetime) dengan tabel AQI.
- Menggunakan left join dengan validate='many_to_one' agar tidak terjadi row explosion.
- Fitur baru: is_polusi = 1 jika $\text{pm25} > 35 \mu\text{g}/\text{m}^3$ (ambang batas WHO).

```

13      2023-10-09      Indigenous Peoples' Day
14      2023-11-10      Veterans Day
... 15      2023-11-23      Thanksgiving Day
16      2023-12-25      Christmas Day

```

```

      nama_libur_en
0      New Year's Day
1      Martin Luther King, Jr. Day
2      Lincoln's Birthday
3      Presidents Day
4      Good Friday
5      Good Friday
6      Truman Day
7      Memorial Day
8      Juneteenth National Independence Day
9      Independence Day
10     Labour Day
11     Columbus Day
12     Columbus Day
13     Indigenous Peoples' Day
14     Veterans Day
15     Thanksgiving Day
16     Christmas Day

```

```

Perjalanan saat hari libur: 141,392

```

Tugas 4: Kamus Data — Dataset Akhir Pipeline

Kamus data lengkap untuk seluruh kolom dataset akhir setelah melalui pipeline ETL termasuk sumber keempat (AQI):

```
# TUGAS 4: kamus_data.md
kamus_md = """"# Kamus Data - Trips Bersih NYC TLC (Januari 2023)

| Kolom | Tipe | Satuan | Sumber | Aturan Validitas | Penanganan Nilai Hilang |
|-----|-----|-----|-----|-----|-----|
| tpep_pickup_datetime | datetime64[ns] | UTC | TLC Parquet | Jan 2023 | Buang baris |
| tpep_dropoff_datetime | datetime64[ns] | UTC | TLC Parquet | > pickup | Buang baris |
| passenger_count | float64 | Orang | TLC Parquet | 1-9 | Isi median per jam |
| passenger_count_hilang | int8 | 0/1 | Turunan | - | - |
| trip_distance | float64 | Mil | TLC Parquet | 0.01-100 | Buang baris |
| trip_distance_capped | float64 | Mil | Turunan (winsorize p99.5) | ≤ p99.5 | - |
| PULocationID | int64 | ID zona | TLC Parquet | 1-263 | Buang baris |
| DOLocationID | int64 | ID zona | TLC Parquet | 1-263 | Buang baris |
| payment_type | int64 | Kode | TLC Parquet | 1-6 | Biarkan |
| fare_amount | float64 | USD | TLC Parquet | ≥ 0 | Biarkan |
| tip_amount | float64 | USD | TLC Parquet | ≥ 0 | Biarkan |
| total_amount | float64 | USD | TLC Parquet | > 0 | Buang baris |
| durasi_menit | float64 | Menit | Turunan (dropoff - pickup) | 1-180 | Buang baris |
| jam | int32 | 0-23 | Turunan | - | - |
| jam_mulai | datetime64[ns] | UTC dibulatkan per jam | Turunan | - | - |
| borough_naik | object | Nama | Zona CSV (join PULocationID) | Diketahui TLC | Left join (NaN =
| zona_naik | object | Nama | Zona CSV | - | Left join |
nama_namayanan | object | Izan | Kita tarif referensi | - | Left join |
print("✅ lineage.csv diperbarui.")

...
✅ kamus_data.md tersimpan.
✅ lineage.csv diperbarui.
```

Tugas 5: Dokumen Lineage — Diagram Alur Data

Diagram alur data (data lineage) menggambarkan perjalanan data dari sumber mentah hingga dataset akhir yang siap analitik. Setiap titik transformasi dan pembuangan baris ditandai secara eksplisit.

```
Disimpan: /content/drive/MyDrive/Bigdata/Praktikum3/lapisan_terkurasi/lineage/
# Data Lineage - Pipeline ETL Praktikum 3

...

## Alur Data

[SUMBER 1] NYC TLC Parquet (~3 juta baris)
[SUMBER 2] Zona Lookup CSV (265 baris)
[SUMBER 3] Open-Meteo API (~30 baris/bulan)
[SUMBER 4] OpenAQ API PM2.5 (~30 baris/bulan)
    |
    v
[T-1] Filter null kritis    --> DROP: trip_distance/fare_amount/PULocationID null
    |
    v
[T-2] Imputasi              --> FILL: passenger_count<-median, RatecodeID<-mode, tip<-per-group me
    |
    v
[T-3] Filter tidak valid    --> DROP: distance<=0, fare<0, dropoff<pickup
    |
    v
[T-4] Winsorizing outlier   --> CLIP: fare_amount & trip_distance ke batas IQR
    |
    v
[T-5] Join zona lookup      --> ADD: Borough, Zone
```

Tugas 6: Reproducibility — Restart and Run All

Notebook dirancang untuk lulus Restart and Run All pada runtime yang bersih dengan memenuhi syarat-syarat berikut:

- Semua library diimpor di sel pertama (K-1). Tidak ada import tersembunyi di sel tengah yang menyebabkan NameError saat dijalankan ulang.
- Semua direktori dibuat dengan `mkdir(parents=True, exist_ok=True)` sehingga tidak gagal meski folder sudah ada maupun belum ada.
- Fungsi `unduh_aman()` memeriksa keberadaan file sebelum mengunduh (`if not path.exists()`). Jika file sudah ada dari run sebelumnya, unduhan dilewati — idempoten.
- Tidak ada variabel global yang diasumsikan sudah ada dari sel sebelumnya. Setiap fungsi menerima parameter eksplisit.
- Pipeline inkremental (`pipeline_inkremental()`) memeriksa output sebelum memproses: `if out_path.exists(): skip` — sehingga aman dijalankan ulang tanpa memproses ulang data yang sama.
- Validasi kontrak skema (`validasi()`) dijalankan di akhir `pipeline()` sebelum menyimpan output — jika ada ketidaksesuaian skema, pipeline gagal dengan pesan yang jelas daripada menyimpan data korup.
- Penanganan `compression='gzip'` secara eksplisit pada `pd.read_csv()` untuk `taxi_zone_lookup.csv` di semua tempat yang membaca file tersebut (termasuk di dalam fungsi `pipeline()`).
- `random_state=42` digunakan pada `train_test_split` dan `.sample()` untuk memastikan hasil yang reproducible setiap kali dijalankan.

```
=====
REPRODUCIBILITY CHECK
=====
...

[1] File sumber:
    ✓ yellow_tripdata_2023-01.parquet
    ✗ TIDAK ADA yellow_tripdata_2023-07.parquet
    ✓ taxi_zone_lookup.csv
    ✗ TIDAK ADA cuaca_nyc_2023.parquet
    ✗ TIDAK ADA aqi_nyc_2023.parquet

[2] Output partisi lapisan_terkurasi/tugas/:
    ✗ Belum ada – jalankan pipeline() dulu

[3] File dokumentasi:
    ✓ laporan_kualitas.csv
    ✗ TIDAK ADA kamus_data.md
    ✓ lineage.md

[4] Cek duplikat antar bulan:
    ✗ Error: No objects to concatenate

=====
Reproducibility check selesai.
=====
```

D. STUDI KASUS: DAMPAK PEMBERSIHAN DATA PADA ANALISIS

E.1 Perbandingan Statistik Sebelum dan Sesudah Pembersihan

```
*** === TARIF PER MIL: HUJAN vs TIDAK HUJAN (per Borough) ===
hujan          0          1
borough_naik
Brooklyn       6.943    7.655
Manhattan     10.527   11.408
Queens         5.453    5.636
Unknown        8.726    9.886
Tabel analitik tersimpan: /content/drive/MyDrive/BigData/Praktikum3/lapisan_terkurasi/tabel_anal
Bentuk tabel : (2094, 7)
```

E.2 Analisis Pengaruh Cuaca terhadap Perjalanan Taksi

Dengan menggabungkan flag is_rain dari data cuaca Open-Meteo:

- Rata-rata trip_distance pada hari hujan: [nilai] mil vs hari cerah: [nilai] mil.
- Rata-rata fare_amount pada hari hujan: \$[nilai] vs hari cerah: \$[nilai].
- Jumlah perjalanan pada hari hujan vs cerah menunjukkan [tren naik/turun/sama].

Distribusi total_amount: Sebelum vs Sesudah

